

Approach:

1. I need to implement a token bucket rate limit for each customer . Each customer should start with a full bucket, tokens should refill continuously over time, each allowed request should consume one token, and denied requests should report how long the client must wait until at least one token is available again. The limiter must track state independently per customer_id and must never let the bucket exceed its fixed capacity.

Assumptions:

1. "current_time_ms" is monotonically non decreasing for requests of the same customer.
2. The limiter is in memory and only needs to work for the lifetime of the process.
3. A request consumes exactly 1 token.
4. "remaining" means tokens left immediately after processing the request.
5. If tokens are fractional internally, I will store them as floating-point values but report "remaining" as an integer.
6. "retry_after_ms" should be returned in milliseconds, since the field name and interface explicitly use ms.

Errors found in the Problem Statement:

1. The statement says "retry_after_ms" should equal the number of "**seconds**" the client needs to wait, but the field name is "**retry_after_ms**" and the interface says milliseconds. The correct behaviour is to return milliseconds , not seconds.
2. The suggested approach says to implement a background timer that fires every 1 second and adds tokens at fixed intervals. This is misleading because the actual requirement says tokens must refill continuously based on elapsed time since the last request, not on a fixed interval.
3. The "**production incident**" says customers were rate-limited on their very first request because the bucket should have started full. That implies the intended initial state is a full bucket, so any example or implementation starting empty is wrong.
4. The problem text is otherwise consistent about a 100-token capacity and 10 tokens per second refill, but the example descriptions must always be interpreted with continuous refill logic, not discrete per second replenishment.

My approach:

I will keep a per-customer record containing:

- A. current token count as a float,
- B. last update timestamp in milliseconds.

On every **check(customer_id,current_time_ms)** call:

1. If the customer is new, initialize the bucket to full capacity and set the last timestamp to **current_time_ms**.
2. Compare elapsed time since the last update.
3. Add **elapsed_ms/1000 * refill_rate** tokens.

4. Cap tokens at **capacity**.
5. If there is at least 1 token, allow the request, subtract 1 token, and return **allowed=True**, the new remaining token count, and **retry_after_ms=0**.
6. If there is less than 1 token, deny the request and compute the time until 1 token becomes available.
 - **tokens_needed=1-current_tokens**
 - **seconds_needed=tokens_needed/refill_rate**
 - **Convert to milliseconds and round up**